LEVEL 12

Chapter 13: Architecture Patterns

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026

Chapter 13: Architecture Patterns — The Complete Catalog

← *Chapter 12: Cloud AWS* · *Index* · *Next* → *Chapter 14: Interview Framework & Communication*

This chapter is a reference catalog. Patterns already deep-dived earlier get a compact summary card with a pointer back; patterns not yet covered (token bucket/leaky bucket mechanics, consistent hashing recap, strangler, sidecar, ambassador, service mesh, fan-out) get the full treatment here.

13.1 Patterns You've Already Learned (Summary Cards)

Pattern	One-line problem	One-line solution	Full treatment
Load balancing	One server can't handle all traffic	Distribute requests across many identical servers	Chapter 4.1
Cache-aside / Read-through / Write-through / Write-back	Repeated DB reads are slow/expensive	Serve hot data from memory, with different consistency/ latency trade-offs per write strategy	Chapter 4.5
CQRS	Read and write workloads have conflicting optimization needs	Separate write model (commands) from read model (queries), sync via events	Chapter 6.8
Event Sourcing	Need full history/audit trail, not just current state	Store the sequence of events, derive state by replay	Chapter 6.8
Saga	Can't use ACID transactions across independently-owned service databases	Chain of local transactions + compensating actions on failure	Chapter 6.8
Outbox	DB write and event publish need to be atomic, but are two different systems	Write the event to an outbox table in the same DB transaction, publish it asynchronously afterward	Chapter 6.8
CDC (Change Data Capture)	Need to react to DB changes without coupling the app to every consumer	Tail the database's transaction log (Debezium) as a stream of change events	Chapter 6.8

Pattern	One-line problem	One-line solution	Full treatment
Pub/Sub / Event-Driven Architecture	Multiple independent systems need to react to the same event, differently	Publish once, many independent subscribers each consume their own copy	Chapter 7.1
Bulkhead	One failing dependency exhausts shared resources, breaking unrelated features	Isolate resource pools (threads/connections) per dependency	Chapter 6.3
Circuit Breaker	Repeatedly calling a failing dependency wastes resources and delays recovery	Trip open after a failure threshold; fail fast; periodically test recovery	Chapter 6.3
Retry (with backoff + jitter)	Transient failures shouldn't need manual intervention	Retry with exponential backoff and jitter, capped attempts, only for idempotent operations	Chapter 6.2
Consistent Hashing	Naive `hash % N` reshuffles nearly all keys when node count changes	Ring-based hashing; adding/removing a node only remaps ~1/N of keys	Chapter 5.4
Sharding	A single database can't hold the data or sustain the write throughput	Horizontally partition rows across multiple database instances	Chapter 5.4
Leader Election / Consensus	Multiple nodes need to agree on who's "in charge" without conflicting	Raft/Paxos-based consensus (via Zookeeper/etcd in practice)	Chapter 6.5

13.2 Rate Limiting Algorithms — Token Bucket vs. Leaky Bucket, Precisely

Both showed up conceptually in Chapter 6.3 and 9.5 — here's the mechanical difference, which is a genuinely common follow-up interviewers ask once you say "I'd add rate limiting."

Token Bucket:
- A bucket holds up to `N` tokens, refilled at a steady rate `R` tokens/second.
- Each incoming request consumes one token; if the bucket is empty, the request is rejected (or queued).
- **Key property: allows bursts.** If the bucket has been filling up unused (client was idle), a client can suddenly send up to `N` requests instantly before being throttled — because those tokens accumulated.

Leaky Bucket:
- Requests enter a queue (the "bucket") and are processed (leak out) at a strictly constant rate, regardless of how bursty the arrivals are.
- If the queue is full, new requests are dropped/rejected.
- **Key property: smooths output to a constant rate**, no matter how bursty the input — the trade-off is added latency for queued requests during a burst, since they wait their turn rather than being processed immediately.

	Token Bucket	Leaky Bucket
Allows bursts	Yes, up to bucket size	No — strictly smooths to a constant rate
Output rate	Variable (up to burst size, then throttled)	Constant
Best for	APIs where occasional bursts are fine (most public APIs)	Systems where a strictly steady downstream processing rate matters (protecting a fragile downstream resource)
Common implementation	Redis with `INCR` + TTL, or a Lua script for atomicity	A queue + fixed-rate worker

> **Interview question:** "Why would you choose token bucket over leaky bucket for a public API rate limiter?"
> **Ideal senior answer:** "Real client traffic isn't smooth — a legitimate client might batch a handful of requests together after being idle, and leaky bucket would either queue and delay all of them or reject the burst outright, which is a worse experience for well-behaved clients. Token bucket tolerates that natural burstiness up to the bucket size while still enforcing a long-run average rate via the refill rate. I'd reach for leaky bucket specifically when I need to protect a downstream system that genuinely can't handle *any* burst — e.g., smoothing writes into a rate-sensitive third-party API with a hard per-second cap."

Sliding window counters (mentioned in Chapter 9.5): a middle ground between simple fixed-window counters (which allow up to 2x the limit right at a window boundary — a burst at 11:59:59 plus a burst at 12:00:01 both fit within their own windows but double the effective rate for that one second) and full token/leaky bucket precision — weight the previous window's count proportionally as it slides out. Redis's sorted-set-based sliding window log, or a weighted-count approximation, are the two common real implementations.

13.3 Fan-Out

The pattern: one event/write triggers many downstream operations. Two flavors, and the distinction is a real interview topic (it came up in Chapter 3's Instagram example):

- **Fan-out on write:** when an event happens, immediately push it to every relevant destination (e.g., a new post is written into every follower's feed cache immediately). Reads become cheap (feed is pre-computed); writes become expensive and proportional to fan-out size (a celebrity with 100M followers means one post triggers 100M writes).
- **Fan-out on read:** don't pre-compute anything at write time; instead, compute the result by pulling from all relevant sources at read time (e.g., build a feed by querying the latest posts from everyone you follow, on demand). Writes stay cheap and constant; reads become expensive and proportional to how much needs to be aggregated.

The standard resolution at scale: hybrid. Fan-out on write for the common case (normal users with a bounded follower count), fan-out on read (or a separate, on-the-fly merge) for outlier "hot" producers (celebrities) whose write-time fan-out cost would be disproportionate. This exact trade-off, and the hybrid resolution, is directly reusable across the feed-based problems in Chapter 17 (Twitter, Instagram) and the notification/fan-out problems in Chapter 16/17.

13.4 Strangler Fig Pattern

Problem: you need to migrate a large legacy monolith to a new architecture (microservices, a rewrite), but a big-bang rewrite is extremely risky — it delays any value delivery until the entire thing is done, and the "flip the switch" cutover is a single high-stakes moment where anything can go wrong with no gradual rollback.

Solution: incrementally replace pieces of the legacy system, routing traffic for a given capability to the new implementation once it's ready, while everything not yet migrated continues to be served by the old system — typically via a routing layer (a reverse proxy/gateway) that decides, per request, whether it goes to "old" or "new." Over time, the new system "strangles" the old one, capability by capability, until nothing routes to the legacy system anymore and it can be decommissioned.

Why it's the correct answer whenever an interviewer asks "how would you migrate this legacy system": it de-risks the migration into many small, independently reversible steps, each of which delivers value and can be validated in production before the next piece moves, instead of betting the whole migration on one irreversible cutover.

> **Interview question:** "How would you migrate a 10-year-old PHP monolith handling checkout to a new microservices-based checkout system, with zero downtime and minimal risk?"
> **Ideal senior answer:** "Strangler fig — I wouldn't touch checkout directly first. I'd start by identifying the least risky, most isolated capability to peel off first (say, order history — read-only, low blast radius if something's subtly wrong), stand up a new service for it, and route just that traffic to the new service via the gateway/routing layer, with the old monolith still handling everything else including the traffic if I need to roll back instantly. Once that's proven stable in production, I'd move to progressively riskier capabilities, saving the actual payment-charging path for last, once the pattern is proven and the team has built confidence and tooling for safe incremental cutover. Given this handles real payments, I'd also run both systems in shadow/parallel for the highest-risk piece before fully cutting over — comparing outputs without the new system's output actually being used yet."

13.5 Sidecar, Ambassador, Service Mesh

Sidecar pattern: deploy a helper process alongside your main application container, in the same pod/host, sharing its lifecycle — handling cross-cutting concerns (logging, monitoring, TLS, retries) *outside* your application code, so every service gets this behavior consistently without each team reimplementing it in every language/framework. **This is exactly what an Envoy proxy sidecar in Istio does** — you've likely already seen this shape on EKS even if not by this name.

Ambassador pattern: a specific flavor of sidecar focused on proxying *outbound* calls to external services — the sidecar handles retries, circuit breaking, and protocol translation for calls going out to other services, so the application code just calls "localhost" and the ambassador handles the network complexity.

Service mesh: a dedicated infrastructure layer (typically built from sidecars deployed next to every service — the "data plane" — plus a central "control plane" configuring them, e.g., **Istio**, **Linkerd**) that handles service-to-service communication concerns uniformly across an entire microservices fleet: mutual TLS between services, traffic routing/canary/shadow deployments, retries and circuit breaking, and rich per-service observability (request success rate, latency, traffic volume) — all *without* any of it living in application code.

	Without a service mesh	With a service mesh
mTLS between services	Each service implements it (or it's skipped)	Automatic, uniform, enforced

	Without a service mesh	With a service mesh
Retries/ circuit breaking	Implemented per-service, often inconsistently (different libraries per language)	Uniform policy, configured centrally, enforced at the proxy layer
Canary/ traffic-shifting	Custom application logic or load balancer config	Declarative traffic-split rules
Cost	Less infrastructure, more per-service engineering effort	More infrastructure/operational complexity (another thing to run and understand), significant latency/resource overhead per hop

> **Interview question:** "Would you introduce a service mesh into your current EKS-based microservices setup?"
> **Ideal senior answer:** "It depends on the actual pain point, not the technology's popularity. If teams are independently and inconsistently implementing retries, mTLS, and observability across a growing number of services in different languages, and that inconsistency is causing real incidents or slowing teams down, a service mesh centralizes and standardizes that — genuinely valuable at, say, 20+ services with multiple teams. Below that scale, or if the team is small and consistent enough to enforce these patterns via shared libraries instead, I'd hold off — a service mesh adds a real operational and latency cost (every call now has two extra proxy hops), and 'we might need it eventually' isn't sufficient justification to pay that cost today. This is the same 'earn the complexity' principle as microservices themselves."

Chapter 13 Interview Drill

1. Precisely distinguish token bucket from leaky bucket, including which one allows bursts.
2. Explain the hybrid fan-out-on-write/fan-out-on-read resolution for a celebrity-follower feed problem.
3. Walk through a strangler fig migration plan for a legacy monolith, in the correct risk order.
4. What does a service mesh's sidecar handle that would otherwise live in application code?
5. Give one concrete reason to *not* adopt a service mesh yet for a 6-service system.

Next → Chapter 14: Interview Framework & Communication — the universal 10-step framework, exactly what to say at each step, senior vs. staff expectations, and the full list of common mistakes.